System Modeling with UML

Part 2: Class · Activity · State Machine Diagrams

By. Soon Phei Tin

Based on Sommerville, Software Engineering (9th ed.) and Fowler, UML Distilled (3rd ed.)

5

Structural Models

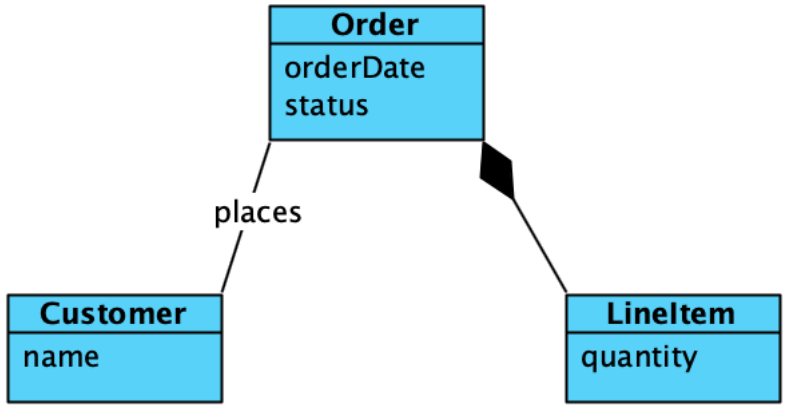
Class Diagrams

What Is a Domain Model?

- **A domain model** is a class diagram that describes the **concepts in the problem space** (the real-world things, their attributes, and how they relate) without any software design decisions.
- It deliberately leaves out visibility, attribute types, and full operation signatures: those are *design* decisions about how the software will be built, not facts about the problem domain itself.
- **A design class diagram** is what the domain model becomes once those decisions are made -- the same classes and relationships, now carrying the visibility, types, and operation signatures the software actually needs.

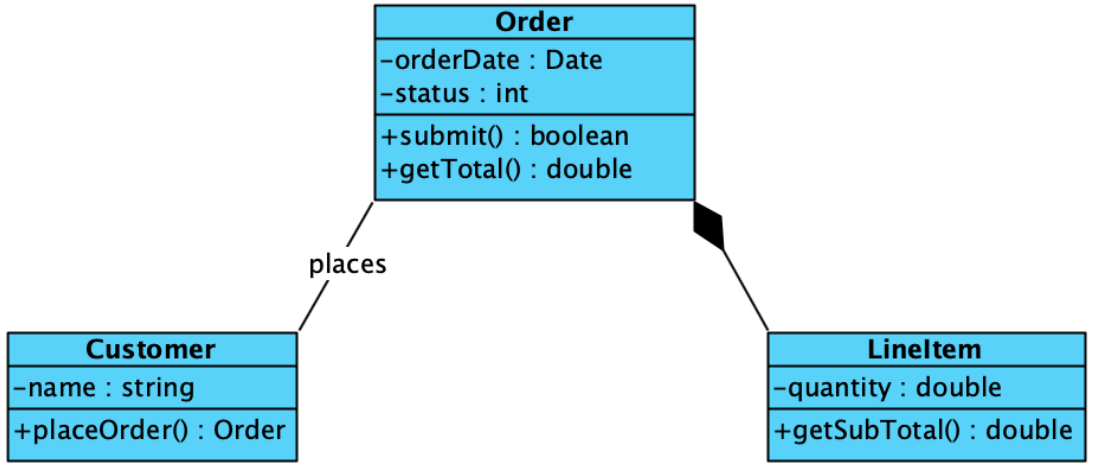
Domain Model vs. Design Class Diagram

Domain Model (conceptual)



No visibility, no types, no operations -- captured during requirements analysis.

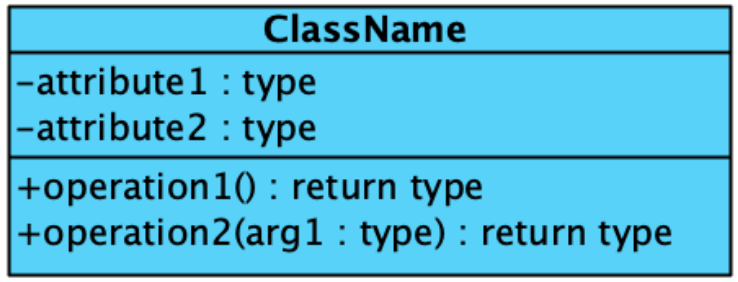
Design Class Diagram



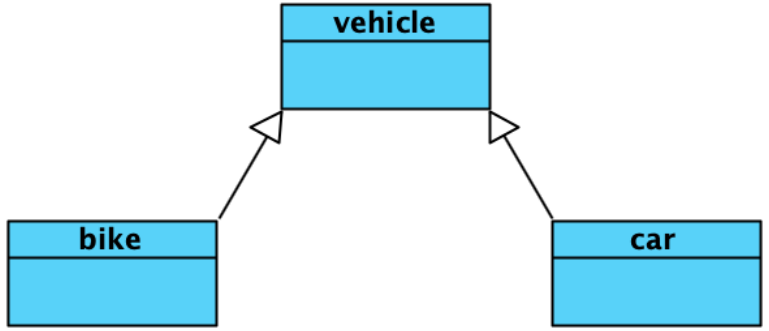
The software classes that implement it. Adds visibility, attribute types, and full operation signatures -- captured during design.

Same classes, same relationships -- design adds visibility, types, and operations.

Class Box Notation & Four Relationships



Class box



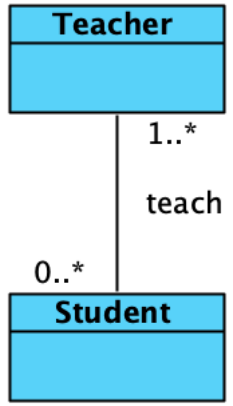
Generalization

- Class box — name, attributes, operations.
- Generalization — hollow triangle to the parent.
- Relationships between classes:
  - Association — a relationship, multiplicity at each end.
  - Aggregation — hollow diamond; parts can outlive the whole.
  - Composition — filled diamond; parts die with the whole.

Association

- Class A “uses” Class B **OR** Class A “knows about” Class B
- The most general relationship: two classes are connected, and objects of one can reference objects of the other. Neither owns the other, and both have independent lifecycles.
- **Example:** a Teacher and a Student. A teacher teaches many students, and a student has many teachers, but neither belongs to the other.

```java
class Teacher {
    private List<Student> students; // refers to students, doesn't own them
}
```

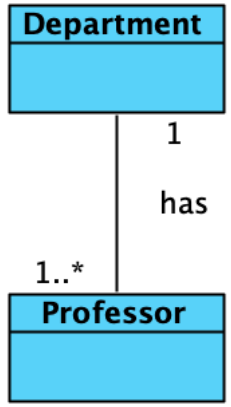


Aggregation

- Class A “has-a” Class B (weak relationship)
- A whole–part relationship where the **part can exist independently of the whole**. The whole groups the parts, but destroying the whole doesn’t destroy the parts, and a part can be shared among several wholes.
- **Example:** a Department and its Professors. If the department closes, the professors still exist and can join another department.

```java
class Department {
    private List<Professor> professors;

    Department(List<Professor> professors) {
        this.professors = professors; // parts created elsewhere, passed in
    }
}
```

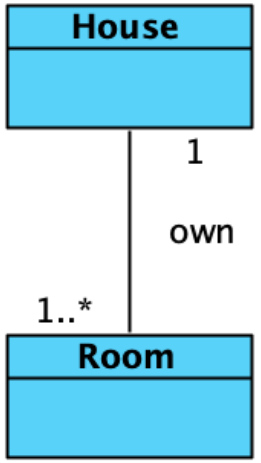


Composition

- Class A “own” Class B (strong relationship)
- A whole–part relationship where the **part cannot exist without the whole**. The whole is responsible for creating and destroying its parts, and a part belongs to exactly one whole at a time.
- **Example:** a House and its Rooms. A room doesn’t exist apart from its house, and demolishing the house destroys the rooms.

```
class House {
    private final List<Room> rooms = new ArrayList<>();

    House(int roomCount) {
        for (int i = 0; i < roomCount; i++) {
            rooms.add(new Room()); // whole creates its parts
        }
    }
    // Room objects are never exposed for reuse elsewhere
}
```



Association vs. Aggregation vs. Composition

Relationship	In Java terms	Lifecycle
Association — plain line	One class holds a reference to another.	Independent lifecycles.
Aggregation — hollow diamond	The whole holds the part as a property field.	Part can outlive the whole.
Composition — filled diamond	The whole creates and owns the part.	Delete the whole, the part goes too.

Question

- What are the differences between Association and Aggregation?

Try It — Requirements for Association, Aggregation, Composition

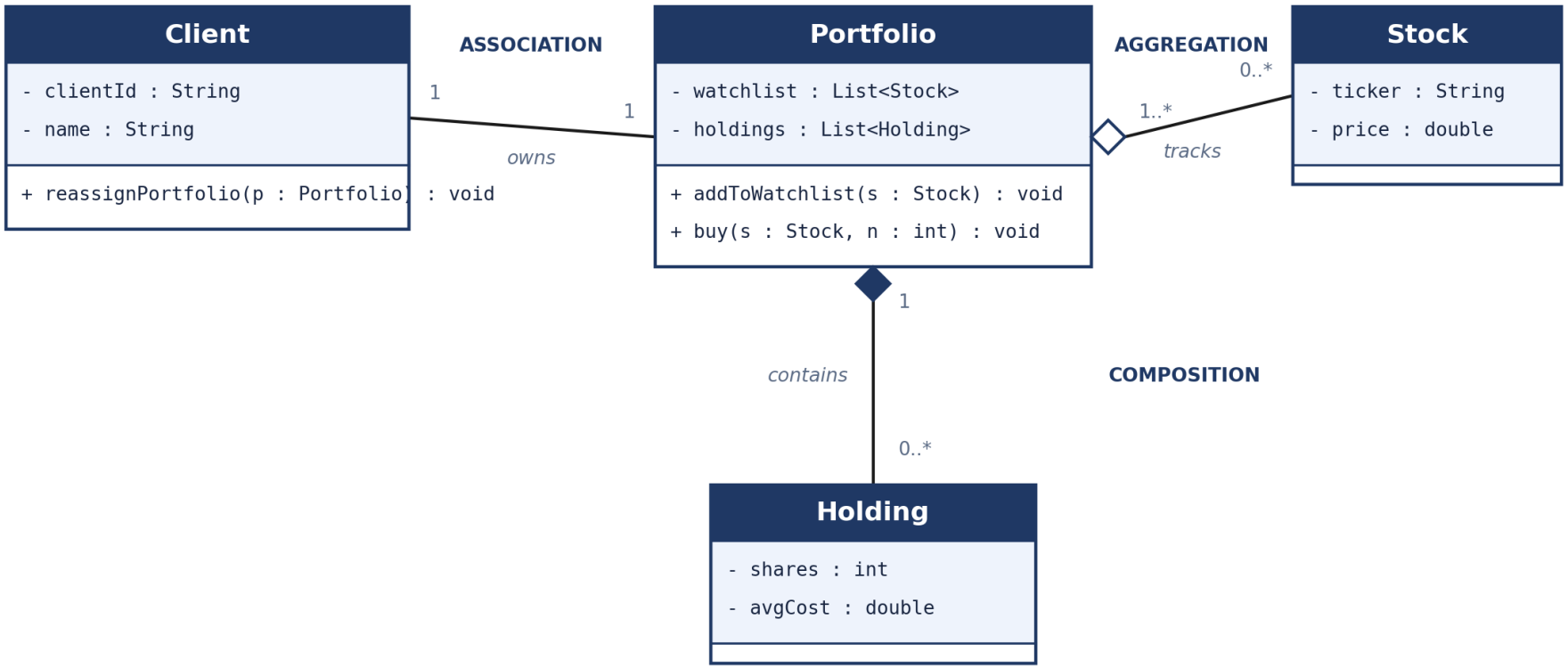
“A Client owns one Portfolio. The Portfolio object is created and assigned to the Client elsewhere in the system when the account is opened, and a Client can later be reassigned to a different Portfolio without either object being destroyed.”

“A Portfolio tracks the Stocks on its watchlist. Each Stock is a security traded on an exchange; it is added to and removed from the watchlist over time, and it continues to exist -- and to be tracked by other portfolios -- even after this Portfolio stops watching it.”

“A Portfolio contains Holdings, one per stock position actually owned. Each Holding is created by the Portfolio itself when a purchase is recorded, and is destroyed the moment the Portfolio is closed, because a Holding has no meaning apart from the Portfolio it belongs to.”

- **Client — Portfolio:** a reference held elsewhere, independent lifecycles → **association**.
- **Portfolio ◇ Stock:** the part outlives the whole → **aggregation**.
- **Portfolio ◆ Holding:** the part cannot outlive the whole → **composition**.

Client / Portfolio / Stock / Holding — Class Diagram



Association (Client–Portfolio), aggregation (Portfolio ◇Stock), and composition (Portfolio ◆Holding) on the same class.

Association, Aggregation, Composition — in Java

```
public class Client {
    private Portfolio portfolio;   // ASSOCIATION: reference held, not created here
    public Client(Portfolio portfolio) { this.portfolio = portfolio; }
}
public class Portfolio {
    private List<Stock> watchlist = new ArrayList<>();   // AGGREGATION
    private List<Holding> holdings = new ArrayList<>();  // COMPOSITION
    public void addToWatchlist(Stock stock) {
        watchlist.add(stock);      // Stock supplied from outside -- not created here
    }
    public void buy(Stock stock, int shares, double price) {
        Holding h = new Holding(stock, shares, price);  // created HERE
        holdings.add(h);           // lives and dies with this Portfolio
    }
    public void close() {
        holdings.clear();          // every Holding is destroyed with the Portfolio
    }
}
```

A reference passed into the constructor (association) vs. an object built with ‘new’ inside the owner and cleared with it (composition).

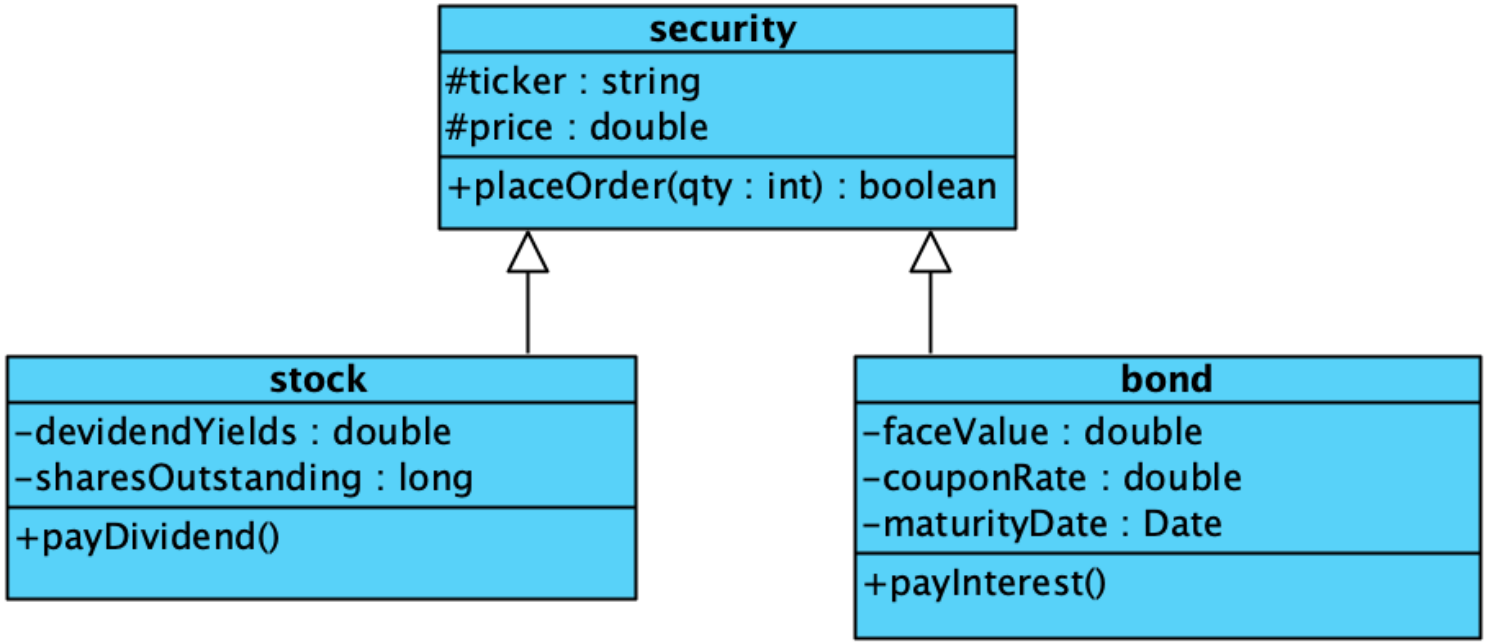
Try It — Requirements for Generalization

“Every tradable instrument in the system is a Security, identified by a ticker symbol and a current price, and every Security can be bought and sold through the same placeOrder() operation.”

“A Stock is a Security that additionally pays dividends and carries a number of outstanding shares. A Bond is a Security that additionally has a face value, a coupon rate, and a maturity date, and pays interest rather than dividends.”

- Stock and Bond are each *“a kind of”* Security -- they share the ticker, price, and placeOrder() behavior, but each adds its own specialized attributes and operations.
- **Stock and Bond generalize to Security**: shown with a hollow triangle pointing from each subclass up to the shared superclass.

Security / Stock / Bond — Generalization



Stock and Bond each inherit ticker, price, and placeOrder() from Security, and add their own attributes and operations.

Identifying Classes

Library Management System — Requirements

“The library needs to keep track of which members currently have which books on loan. A member registers with the library and, once registered, may borrow many books over the course of their membership. Any given book may likewise be borrowed by many different members over its lifetime on the shelf, but never by more than one member at a time. Each time a book is borrowed, the library must record when the loan began, when it falls due, and when the book is actually returned — because a book that is overdue or lost has to be traced back to one specific borrowing event, not just to a member and a book in the abstract.”

Identifying Classes and Attributes: A Practical Method

Nous-verb Analysis

- Filter nouns into classes, attributes, or discards
  - Noun-verb analysis: nouns → candidate classes/attributes; verbs → candidate operations.
  - Keep as a class if it has its **own identity**, its **own data**, and its **own behavior or relationships**.
  - Make it an attribute if it's a **simple value describing something else**.
  - **Discard** if it's redundant, vague, outside the system's scope, or refers to the system itself.
  - **Merge synonyms**: if the text says both "member" and "patron," pick one.
- **Naming: singular CamelCase nouns, pronounceable, never starting with a digit.**

Identifying Relationship and Operations: A Practical Method

Nous-verb Analysis

- Turn verbs into associations or operations
  - Verbs that connect two nouns usually become **associations**: a member *borrows* a book, a member *pays* a fine.
  - Verbs describing something a class does usually become **operations** in a design class diagram: register(member), addBook(book), returnBook().
  - **“Has,” “contains,” “consists of”** often signal aggregation or composition.
  - Constraints in the text become multiplicities: “up to five books at a time” gives Member 1 — 0..5 active Loan.
- In a pure domain model, you’d keep the associations and skip the operations, since operations belong to the design phase

Nous-verb Analysis: Strengths and limitations

The technique's strength is that it's **mechanical and easy to start**: it gives you a first-pass list quickly and keeps you grounded in the requirements' vocabulary. Its weakness is that **natural language is imprecise**:

- The same concept may appear under different words.
- Important concepts may be implied but never named.
- Grammar can mislead: "a payment" is a noun, but "to pay" is a verb for the same idea.

Try It — Identifying Classes in a Shopping Cart

"A registered Customer builds up a ShoppingCart with CartItems before checking out; each CartItem records which Product was chosen and the quantity of it the customer wants."

Noun	Own identity	Own data	Has r/s	Has behavour	Final Result
Customer	Yes	Yes	Yes	Yes	Class
ShoppingCart	Yes	Yes	Yes	Yes	Class
CartItem	Yes	Yes	Yes	-	Class
Product	Yes	Yes	Yes	Yes	Class
quantity	No	Single value	No	No	Attribute
registered	No	Single value	No	No	Attribute. If Customer has an attribute isRegistered. All Customer is registered therefore isRegistered is unnecessary.

Identifying Relationships: A Practical Method

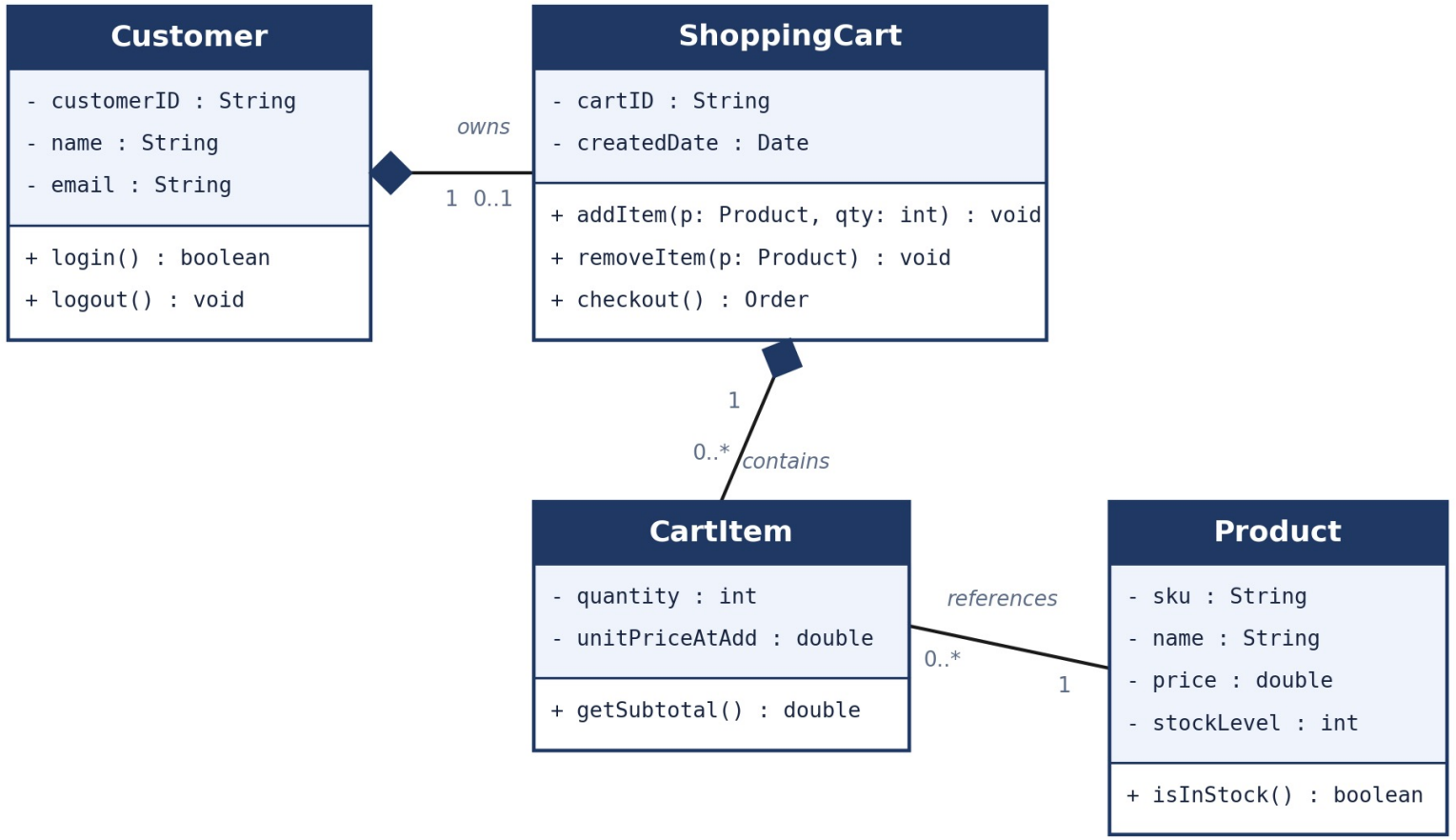
- Extract the verbs that connect two nouns — “builds up”, “records which Product”, “wants” — each one is a candidate operation or relationship.

Verb phase	Relationship	Reasoning
Customer builds up ShoppingCart	Customer ◆ ShoppingCart	The phase is inaccurately describe the r/w. *Customer own ShoppingCart* all the time even if he/she did not add and item. Builds up here indicating the customer adding items over time to fill the shopping cart. *Customer own ShoppingCart* before he can fill it with items.
ShoppingCart with CartItem	ShoppingCart ◆ CartItem	CartItems are part of the ShoppingCart.
CartItem records which Product	CartItem - Product	CartItem represent with Product customer want to buy and what quantity.

Identifying Relationships: A Practical Method

- Fix the multiplicity on each end by asking directly: how many of the first class relate to how many of the second?
- Watch for “is a kind of” (generalization).
- Validate by walking one real scenario end to end across the diagram.

Shopping Cart — Class Diagram



Composition (◆): a CartItem, and the Cart itself, have no meaning once their owner is gone.
Association (plain line): a Product stays in the catalogue whether or not it is in anyone's cart.

Library Management System — Requirements

“The library needs to keep track of which members currently have which books on loan. A member registers with the library and, once registered, may borrow many books over the course of their membership. Any given book may likewise be borrowed by many different members over its lifetime on the shelf, but never by more than one member at a time. Each time a book is borrowed, the library must record when the loan began, when it falls due, and when the book is actually returned — because a book that is overdue or lost has to be traced back to one specific borrowing event, not just to a member and a book in the abstract.”

Library — Identifying Entities

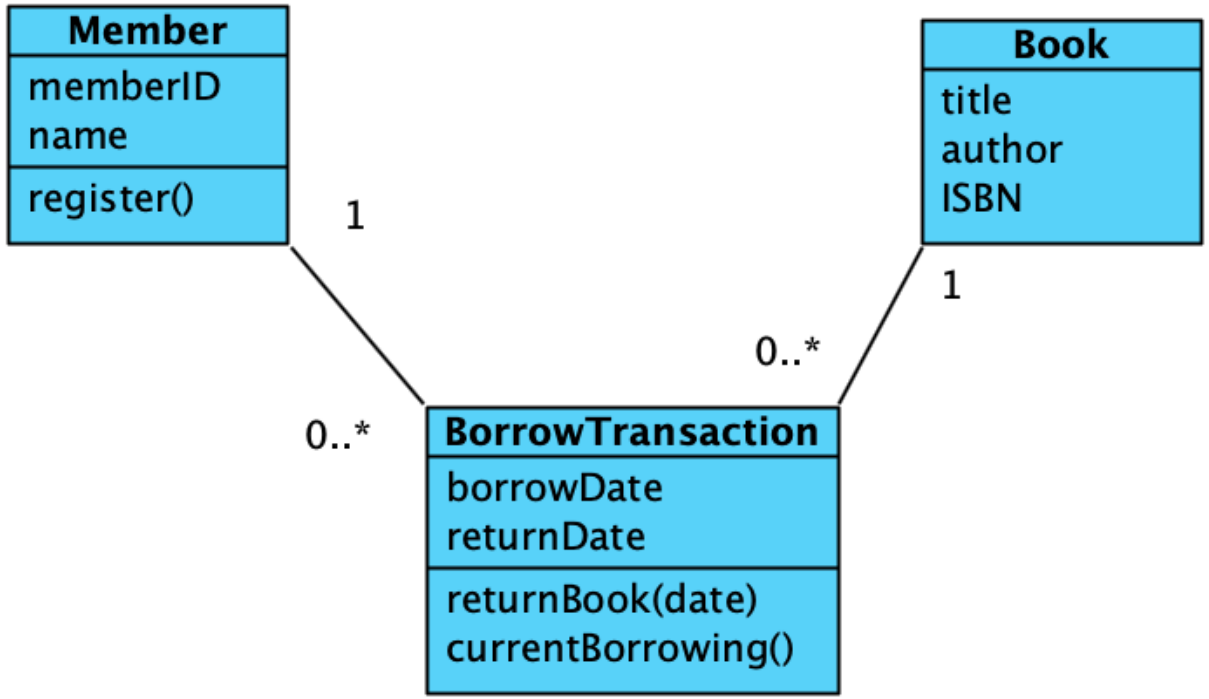
- **Member** — kept: Has identity, data, r/s.
- **Book** — kept: Has identity, data (title, author, ISBN), r/s.
- **BorrowTransaction** — kept: The text explicitly demands that each borrowing be individually traceable (identity). Has data (start date, due date and return date), r/s.
- **Library –** Rejected: The system itself.
- **Membership –** Rejected: Represented by member.
- **shelf, lifetime** — Rejected: Vague, no data or r/s.

Library — Identifying Relationships & Operation

- Verbs: *registers, borrows, records, is returned, traced back, keep track*

Verb phase	Note	Reasoning
member *registers* with the library	Member.register()	
member *borrows* book	Relationship: Member – BorrowTransaction BorrowTransaction - Book	The borrowing is itself the event being modeled
library *records* when the loan began, falls due, returned	Attributes of BorrowTransaction	
book *is returned*	BorrowTransaction.returnBook(date)	
traced back to one borrowing event	Confirms BorrowTransaction is linked to both Member and Book	
keep track of which members currently have which books	Query operation. BorrowTransaction.currentBorrowing()	

Worked Example — Library Management System



BorrowTransaction is its own class (an association class) holding transaction-specific data.

Online Banking System — Requirements (Recap)

- **OBS-01** — a Customer authenticates with a customerID and password → a login() operation.
- **OBS-06–OBS-11** — a Customer transfers funds between accounts; the transfer needs its own reference number, amount, timestamp and status.
- **OBS-11, OBS-13** — every transfer must be traceable to exactly one source Account and one destination Account.
- **OBS-18–OBS-20** — a Bank Administrator manages Accounts and generates reports, but never touches a Transfer directly.
- **Domain knowledge** (beyond the OBS list): a savings account earns interest; a checking account may allow overdraft — two account types.

Online Banking — Identifying Entities

- **Customer, BankAdministrator** — kept: Own identity, data, r/s, operation.
- **Transfer** — kept, as its own class: Own identity, data.
- **Account** — kept, with two specializations, SavingsAccount and CheckingAccount, wherever interest calculation or overdraft authorization actually differ — a generalization. Own data, r/s, operation.
- **customerID, password** — attributes of Customer
- **Reference number, amount, timestamp, status** – attribute of Transfer
- **Interest** – attribute of SavingAccount
- **Overdraft** – attribute of CheckingAccount

Online Banking — Identifying Relationships

- Verbs: *manages, traceable to.*

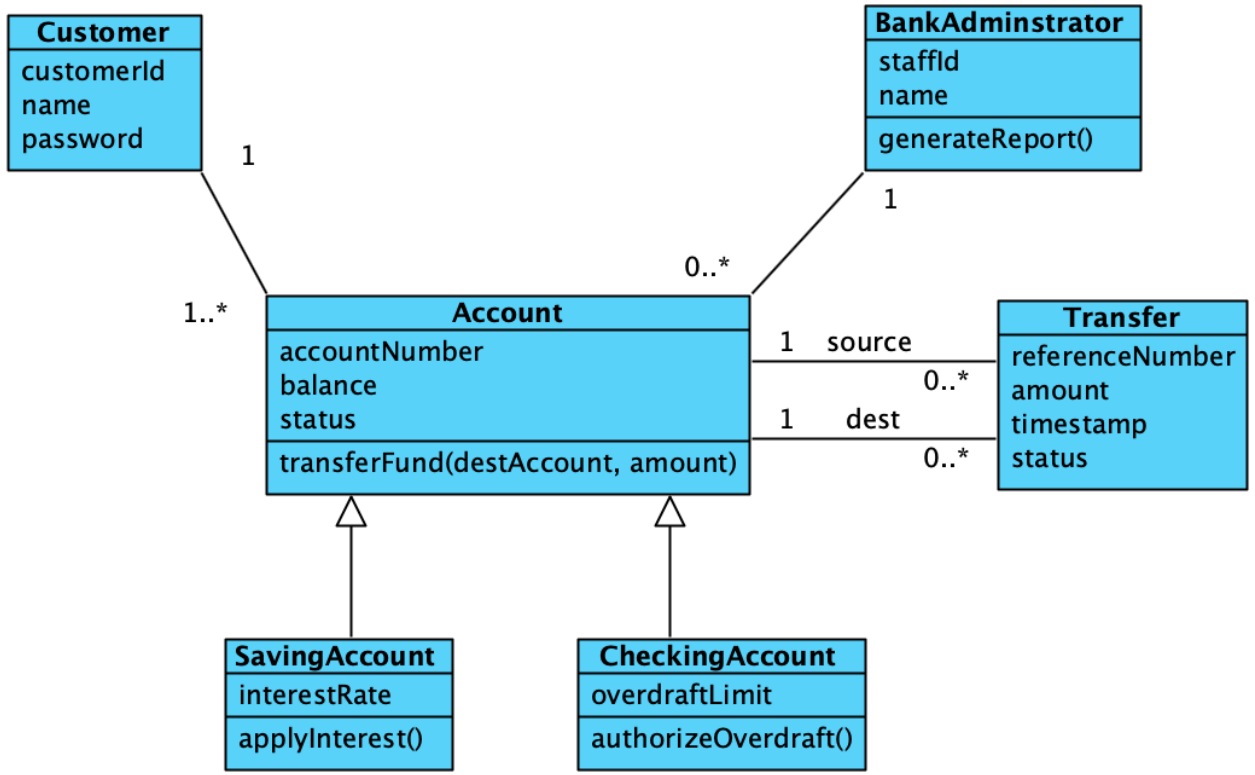
- **Customer —owns→ Account** (1 to 1..*) – Not explicitly mentioned.
- **Administrator manages Account: BankAdministrator —manages→ Account** (1 to 0..*) — neither pair's lifecycle depends on the other.
- **Transfer traceable to one source and one destination Account:** Transfer needs two associations to Account for the very same pair of classes — validating a source and a destination separately — so each end gets a role name: *source* and *destination*.
- SavingsAccount and CheckingAccount generalize Account, inheriting accountNumber, balance and its operations.

Online Banking — Identifying Operations

- Verbs: *authenticate, transfers, generates, earns, allow*

- **Customer.login() —** Customer ***authenticates*** with customerID and password.
- **Account.transferFund()** – Customer ***transfers*** funds between accounts.
- **BankAdministrator.generateReport() –** Administrator ***generates*** reports.
- **SavingAccount.applyInterest() –** Savings account ***earns*** interest.
- **CheckingAccount.applyOverdraft() –** Checking account may ***allow*** overdraft.

Online Banking — Class Diagram



Two role-named associations (source, destination) connect Transfer to Account; SavingsAccount and CheckingAccount generalize Account.

Questions?

Reading notes with worked examples and solved exercises are available alongside this deck.